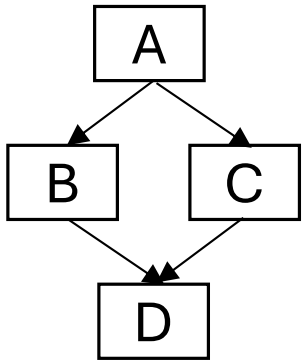
More fork-join

5/12/25

Recall: fork and join

```
A();
fork B();
C();
join;
D();
```



Each time unit, each processor runs a strand w/o unfinished predecessors (if one exists) and finishes it

Both B and C must run after A and before D, but their order is flexible

Which of the following could be printed by the code below?

```
System.out.print(1);
fork System.out.print(2);
System.out.print(3);
System.out.print(4);
join;
```

I. 1234
II. 2134
III. 1324

A. Only I.
B. Only II.
C. Only I. and II.
D. Only II. and III.
E. None of the above

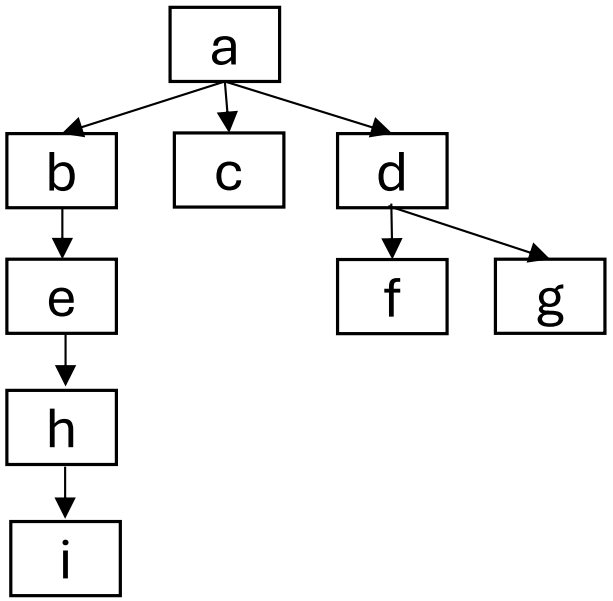
Which of the following could be printed by the code below?

```
System.out.print(1);
fork System.out.print(2);
System.out.print(3);
System.out.print(4);
join;
```

I. 1234
II. 2134
III. 1324

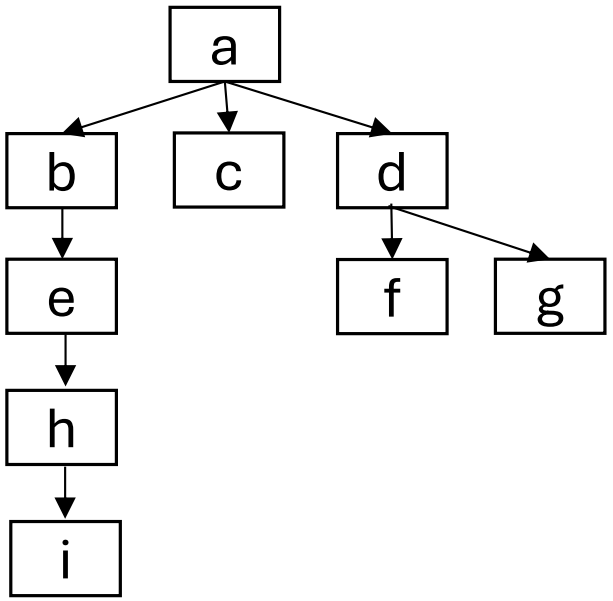
A. Only I.
B. Only II.
C. Only I. and II.
D. Only II. and III.
E. None of the above (I. and III.)

How long will this graph run on 2 processors?



A. Exactly 4 time units
B. Exactly 5 time units
C. Either 4 or 5 time units
D. Either 5 or 6 time units
E. None of the above

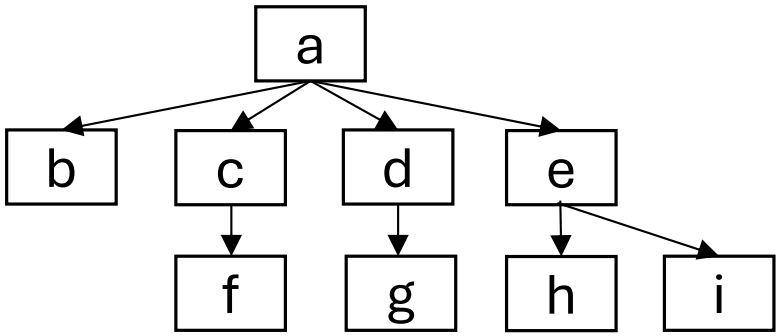
How long will this graph run on 2 processors?



A. Exactly 4 time units
B. Exactly 5 time units
C. Either 4 or 5 time units
D. Either 5 or 6 time units
E. None of the above

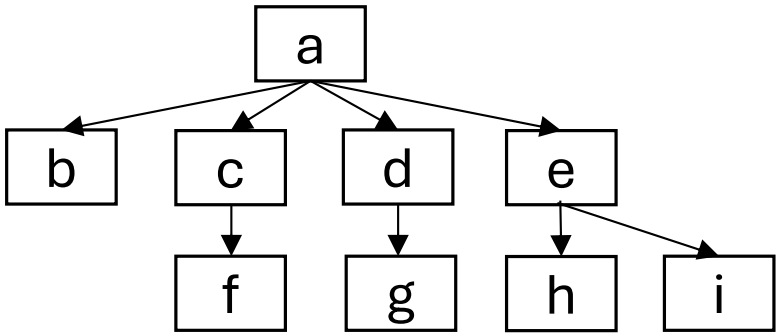
(between 5 and 7)

How long will this graph run on 2 processors?



A. Exactly 4 time units
B. Exactly 5 time units
C. Either 4 or 5 time units
D. Either 5 or 6 time units
E. None of the above

How long will this graph run on 2 processors?



A. Exactly 4 time units
B. Exactly 5 time units
C. Either 4 or 5 time units
D. Either 5 or 6 time units
E. None of the above

Now we're trying to optimize 2 things

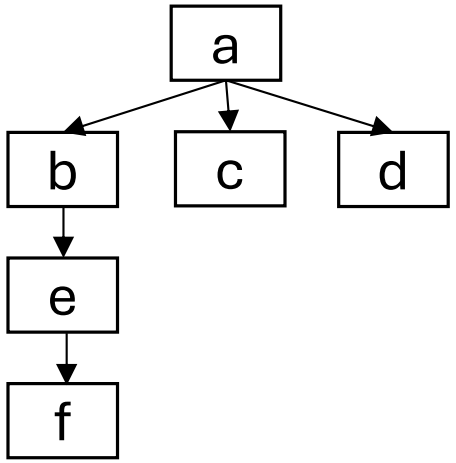
- Work = number of strands (denoted T_1)
- Span = length of longest path (denoted T_∞)

Now we're trying to optimize 2 things

- Work = number of strands (denoted T_1)
- Span = length of longest path (denoted T_∞)

What are the work and span of this graph?

A. 3 and 4
B. 4 and 4
C. 6 and 3
D. 7 and 3
E. None of the above

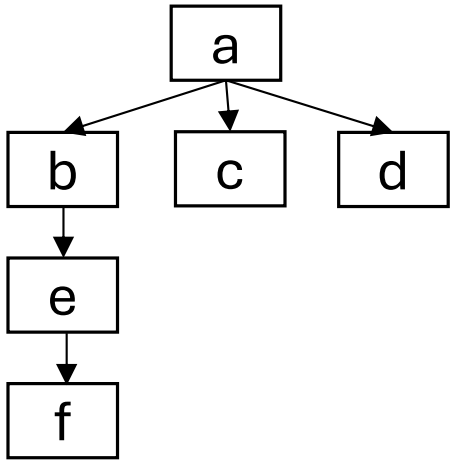


Now we're trying to optimize 2 things

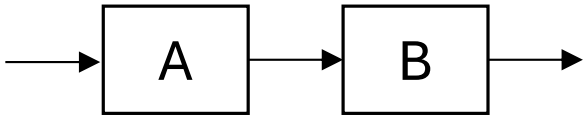
- Work = number of strands (denoted T_1)
- Span = length of longest path (denoted T_∞)

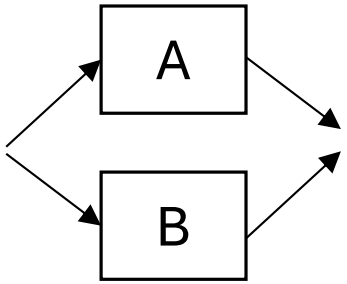
What are the work and span of this graph?

A. 3 and 4
B. 4 and 4
C. 6 and 3
D. 7 and 3
E. None of the above (6 and 4)

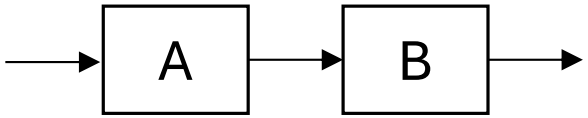


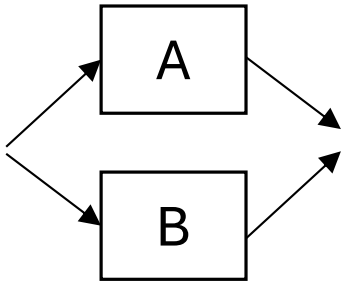
Combining subcomputations





Combining subcomputations

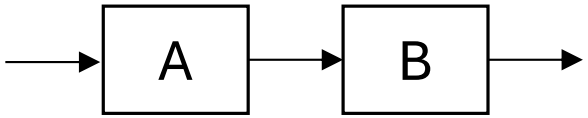


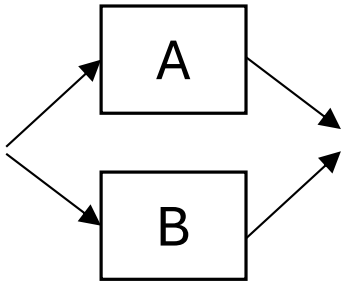


Work: $T_1(A) + T_1(B)$

Work: $T_1(A) + T_1(B)$

Combining subcomputations





Work: $T_1(A) + T_1(B)$

Span: $T_\infty(A) + T_\infty(B)$

Work: $T_1(A) + T_1(B)$

Span: $\max\{ T_\infty(A), T_\infty(B) \}$

What best characterizes the span of this code?

(n is hi-low)

```
int sum(int[] array, int low, int hi) {
        if(hi==low+1)
                return array[low];
        int left = fork sum(array, low, (hi+low)/2);
        int right = sum(array, (hi+low)/2, hi);
        join;
        return left + right;
}
```

A. O(1)
B. O(log n)
C. O(n)
D. O(n log n)
E. None of the above

What best characterizes the span of this code?

(n is hi-low)

```
int sum(int[] array, int low, int hi) {
        if(hi==low+1)
                return array[low];
        int left = fork sum(array, low, (hi+low)/2);
        int right = sum(array, (hi+low)/2, hi);
        join;
        return left + right;
}
```

A. O(1)
B. O(log n)
C. O(n)
D. O(n log n)
E. None of the above

What best characterizes the work of this code?

(n is hi-low)

```
int sum(int[] array, int low, int hi) {
        if(hi==low+1)
                return array[low];
        int left = fork sum(array, low, (hi+low)/2);
        int right = sum(array, (hi+low)/2, hi);
        join;
        return left + right;
}
```

A. O(1)
B. O(log n)
C. O(n)
D. O(n log n)
E. None of the above

What best characterizes the work of this code?

(n is hi-low)

```
int sum(int[] array, int low, int hi) {
        if(hi==low+1)
                return array[low];
        int left = fork sum(array, low, (hi+low)/2);
        int right = sum(array, (hi+low)/2, hi);
        join;
        return left + right;
}
```

A. O(1)
B. O(log n)
C. O(n)
D. O(n log n)
E. None of the above

What best characterizes the work and span of this code?

(n is hi-low)

```
int sum(int[] array, int low, int hi) {
        int total = 0;
        for(int i=low; i < hi; i++)
                total += array[i];
        return total;
}
```

A. O(log n) and O(log n)
B. O(log n) and O(n)
C. O(n) and O(log n)
D. O(n) and O(n)
E. None of the above

What best characterizes the work and span of this code?

(n is hi-low)

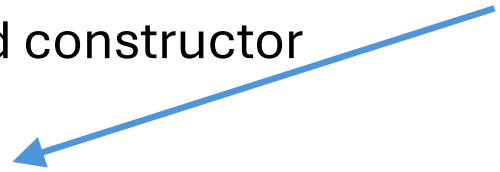
```
int sum(int[] array, int low, int hi) {
        int total = 0;
        for(int i=low; i < hi; i++)
                total += array[i];
        return total;
}
```

A. O(log n) and O(log n)
B. O(log n) and O(n)
C. O(n) and O(log n)
D. O(n) and O(n)
E. None of the above

In actual Java: RecursiveTask

```
class Sum extends RecursiveTask<Integer> {
    //attributes and constructor

    protected Integer compute(){
        if(hi – low < THRESHOLD) return simpleSum(array, low, hi);
        Sum left = new Sum(array, low, (hi+low)/2);
        Sum right= new Sum(array, (hi+low)/2, hi);
        left.fork();
        return right.compute() + left.join();
    }
}
```



In actual Java: RecursiveTask

```
class Sum extends RecursiveTask<Integer> {
   //attributes and constructor

   protected Integer compute(){
      if(hi – low < THRESHOLD) return simpleSum(array, low, hi);
      Sum left = new Sum(array, low, (hi+low)/2);
      Sum right= new Sum(array, (hi+low)/2, hi);
      left.fork();
      return right.compute() + left.join();
   }
}
```

Only recurse to create enough strands

In actual Java: RecursiveTask

```
class Sum extends RecursiveTask<Integer> {
    //attributes and constructor

    protected Integer compute(){
        if(hi – low < THRESHOLD) return simpleSum(array, low, hi);
        Sum left = new Sum(array, low, (hi+low)/2);
        Sum right= new Sum(array, (hi+low)/2, hi);
        left.fork();
        return right.compute() + left.join();
    }
}
```

Note: Only create new strand for one half

In actual Java: main

```
sum = ForkJoinPool.commonPool().invoke(
                        new Sum(array, 0, array.length));
```